

146. LRU Cache

PLATFORM

DIFFICULTY

DATE

Platform: LeetCode

MEDIUM

Solved: July 13, 2021

Input Problem Summary

Design a data structure that supports `get(key)` and `put(key, value)` in $O(1)$ time, evicting the Least Recently Used key when capacity is exceeded.

Approach

1. Calculate `months * salary` for each employee
2. Find the MAX of that computed value using subquery
3. Filter employees where their earnings = max earnings
4. `COUNT(*)` gives number of such employees
5. `MAX(months*salary)` gives the maximum value

Algo

- Initialize HashMap, dummy head, dummy tail, connect `head.next = tail`, `tail.prev = head`.
- `addToFront(node)`: wire node between head and tail using 4 pointer updates.
- `removeNode(node)`: wire `node.prev` to `node.next` using 2 pointer updates.
- `get(key)`: if not in map return -1. Else `removeNode` + `addToFront` + return `node.value`.
- `put(key, value)`: if exists → update value + `removeNode` + `addToFront`. Else → if full, evict `tail.prev` from list and map. Then create new node, `addToFront`, put in map.

Time Complexity & Space Complexity

$O(1)$ — all operations are direct HashMap access + fixed pointer updates.

$O(\text{capacity})$ — map and list both hold at most capacity nodes.

Approach

- Combine a **HashMap** for $O(1)$ key lookup with a **Doubly Linked List** for $O(1)$ insertion and deletion. Most Recently Used node sits just after a dummy head, Least Recently Used sits just before a dummy tail. Every access or insertion moves the node to the front; eviction removes from just before tail.

Observations

- No GROUP BY needed — we want a single row output
- Subquery calculates max earnings independently → used in WHERE to filter
- MAX() and COUNT(*) together on filtered rows gives both answers in one query

Key Insights

- HashMap gives $O(1)$ access to any node; doubly linked list gives $O(1)$ insertion/deletion anywhere.
- Dummy head and tail eliminate null checks — no special casing for empty list or single node.
- Node stores key so that during eviction (`tail.prev`), you know which map entry to delete.
- `removeNode` + `addToFront` together = "mark as most recently used" — reused in both `get` and `put`.

Observations

- `addToFront` needs 4 pointer updates in the right order — do `node.next` and `node.prev` first before touching `head.next`, or you lose the reference to the old `head.next`.
- `removeNode` only needs 2 pointer updates — no need to null out `node.prev` and `node.next`.
- Both `get` and `put` (update case) share the same "move to front" logic — could be extracted to a helper `moveToFront()`.
- `map.size() == capacity` check is used (not `>`) since we add to map after the eviction.

146. LRU Cache

Solved

MediumTopicsCompanies

Design a data structure that follows the constraints of a **Least Recently Used (LRU) cache**.

Implement the `LRUCache` class:

- `LRUCache(int capacity)` Initialize the LRU cache with **positive** size `capacity`.
- `int get(int key)` Return the value of the `key` if the key exists, otherwise return `-1`.
- `void put(int key, int value)` Update the value of the `key` if the key exists. Otherwise, add the `key-value` pair to the cache. If the number of keys exceeds the `capacity` from this operation, **evict** the least recently used key.

The functions `get` and `put` must each run in $O(1)$ average time complexity.

Example 1:

Input
["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]

Output
[null, null, null, 1, null, -1, null, 3, 4]

Explanation
LRUCache lruCache = new LRUCache(2);
lruCache.put(1, 1); // cache is {1=1}
lruCache.put(2, 2); // cache is {1=1, 2=2}
lruCache.get(1); // return 1
lruCache.put(3, 3); // LRU key was 2, evicts key 2, cache is {1=1, 3=3}
lruCache.get(2); // returns -1 (not found)
lruCache.put(4, 4); // LRU key was 1, evicts key 1, cache is {4=4, 3=3}
lruCache.get(3); // return 3 (not found)
lruCache.get(4); // return 4

Constraints:

- $1 \leq \text{capacity} \leq 3000$
- $0 \leq \text{key} \leq 10^4$
- $0 \leq \text{value} \leq 10^5$
- At most 2×10^5 calls will be made to `get` and `put`.

over
196

Seen this question in a real interview before? 1/6

YesNo

Accepted 26,78,066/5.6M Acceptance Rate 47.7%

23.3K456418 Online

Accepted

All Submissions

Accepted 25 / 25 testcases passed
Submitted at Jul 13, 2026 18:58
AnalysisSolution

Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert editorials —>

Runtime

44 ms Beats 83.70%

Memory

130.63 MB Beats 86.14%

Code

Java

```
1 class LRUCache {
2
3     class Node {
4         int key, value;
5         Node prev, next;
6         Node(int key, int value){
7             this.key = key;
8             this.value = value;
9         }
10    }
11
12    Node head, tail;
13    Map<Integer, Node> map;
14    int capacity;
15
16    public LRUCache(int capacity) {
17        this.capacity = capacity;
18        map = new HashMap<>();
19        head = new Node(0, 0);
20        tail = new Node(0, 0);
21        head.next = tail;
22        tail.prev = head;
23    }
24
25    private void addToFront(Node node){
26        node.next = head.next;
27        head.next.prev = node;
28        head.next = node;
29        head.prev = node;
30    }
31
32    private void removeNode(Node node){
33        node.prev.next = node.next;
34        node.next.prev = node.prev;
35    }
36
37    public int get(int key) {
38        if(!map.containsKey(key)) return -1;
39        Node node = map.get(key);
40        removeNode(node);
41        addToFront(node);
42        return node.value;
43    }
44
45    public void put(int key, int value) {
46        if(map.containsKey(key)){
47            Node node = map.get(key);
48            node.value = value;
49            removeNode(node);
50            addToFront(node);
51        } else {
52            Node node = new Node(key, value);
53            if(map.size() == capacity){
54                map.remove(tail.prev.key);
55                removeNode(tail.prev);
56            }
57            addToFront(node);
58            map.put(key, node);
59        }
60    }
61 }
```

More challenges

460. LRU Cache588. Design In-Memory File System604. Design Compressed String Iterator

Write your notes here

Select related tags0/5

418 Online

Code

JavaAuto

```
1 class LRUCache {
2
3     class Node {
4         int key, value;
5         Node prev, next;
6         Node(int key, int value){
7             this.key = key;
8             this.value = value;
9         }
10    }
11
12    Node head, tail;
13    Map<Integer, Node> map;
14    int capacity;
15
16    public LRUCache(int capacity) {
17        this.capacity = capacity;
18        map = new HashMap<>();
19        head = new Node(0, 0);
20        tail = new Node(0, 0);
21        head.next = tail;
22        tail.prev = head;
23    }
24
25    private void addToFront(Node node){
26        node.next = head.next;
27        head.next.prev = node;
28        head.next = node;
29        head.prev = node;
30    }
31
32    private void removeNode(Node node){
33        node.prev.next = node.next;
34        node.next.prev = node.prev;
35    }
36
37    public int get(int key) {
38        if(!map.containsKey(key)) return -1;
39        Node node = map.get(key);
40        removeNode(node);
41        addToFront(node);
42        return node.value;
43    }
44
45    public void put(int key, int value) {
46        if(map.containsKey(key)){
47            Node node = map.get(key);
48            node.value = value;
49            removeNode(node);
50            addToFront(node);
51        } else {
52            Node node = new Node(key, value);
53            if(map.size() == capacity){
54                map.remove(tail.prev.key);
55                removeNode(tail.prev);
56            }
57            addToFront(node);
58            map.put(key, node);
59        }
60    }
61 }
```

TestcaseTest Result

Ln 61, Col 2